ACADEMIC RESEARCH REPORT | SUPABASE POSTGRESQL 15+ | DUAL-ENGINE ARCHITECTURE | OPEN-SOURCE STT | 59/59 AUTOMATED TESTS VERIFIED

QueryMind 2.0: Voice-Enabled DBMS Integration

Comprehensive Architectural Blueprint, Speech-to-Text Fallbacks, Cloud Relational Migration, Advanced Query Governance, and Autonomous Development Loop

Document Reference: QM2-INT-DEEP-2026 **Branch Status:** `querymind-2.0` (59/59 Tests Passing) **Primary Target:** Supabase PostgreSQL & Web Speech API / Vosk

EXECUTIVE SUMMARY

QueryMind 2.0 upgrades the existing natural-language-to-SQL web application with voice input capabilities and a comprehensive **Supabase (PostgreSQL 15+)** cloud backend. Crucially, the platform elevates Database Management Systems (DBMS) education as its central anchor, introducing query execution plan analysis (`EXPLAIN QUERY PLAN`), relational schema introspection, dynamic Mermaid Entity-Relationship (ER) generation, and database-level Row-Level Security (RLS).

This report builds upon the verified starting point established on the `querymind-2.0` branch—including the in-browser Web Speech API controller, the dual-engine database adapter with zero-config SQLite fallback, AST security filtering via SQLGlot, and the 59 automated test cases passing in the repository. We detail open-source speech recognition fallbacks, migration methodologies, security governance, and an autonomous test-driven development loop adhering to a strict *"zero unverified claims"* standard.

1. Open-Source Speech-to-Text (STT) Options & Fallback Architecture

For voice input, QueryMind's browser UI currently uses the **Web Speech API (SpeechRecognition)** to capture microphone audio directly inside client JavaScript (`static/js/voice_sql.js`). The Web Speech API is built into modern Chromium-based browsers (Chrome, Edge) and enables developers to incorporate voice data into web applications without external JavaScript bundles. It provides an immediate, low-latency speech-recognition interface.

However, browser support can be inconsistent across platforms (Firefox provides only partial support behind experimental flags, Safari limits custom recognition, and network firewalls may block cloud speech endpoints). To guarantee cross-browser resilience and offline capabilities, we evaluate open-source ASR models and fallback toolkits:

Speech Engine	License	Offline / Local	Browser Support	Operational Trade-offs & Role
Web Speech API	W3C Standard	No (Requires network)	Chrome, Edge, Chromium	**Primary Client Tier:** Zero download size; real-time streaming tokens; highly responsive on desktop and mobile Chrome/Edge.
OpenAI Whisper (Python)	MIT	Yes (Local CPU/GPU)	Universal (Server Endpoint)	**High-Accuracy Server Backup:** Encoder-decoder transformer trained on 680k hours; robust to accents; ideal for uploaded audio streams via `POST /api/transcribe`.
Vosk (Alpha Cephei)	Apache 2.0	Yes (Lightweight WASM)	Modern Browsers via WASM	**Offline Client/Server Backup:** Compact ~50MB acoustic models; runnable in-browser via WebAssembly or on Flask backend with sub-second inference.
Coqui STT / Kaldi	MPL 2.0 / Apache	Yes	Server Only	Legacy engines; Coqui is discontinued and Kaldi requires complex pipeline compilation. Replaced by Whisper and Vosk in modern architectures.

> **CORE OPERATING RULE: PRE-EXECUTION INSPECTION**
>
> In QueryMind 2.0, the microphone functions strictly as an **input capture mechanism**. Spoken audio is transcribed into the query textarea, where users can inspect, modify, or regenerate the SQL. **Queries are never automatically executed on speech termination.** The user retains authoritative control to click `[Run Query]` or `[Explain Plan]`.

2. Supabase / PostgreSQL Integration & Security Architecture

Integrating Supabase equips QueryMind with an enterprise-grade cloud **PostgreSQL 15+** object-relational database. PostgreSQL is celebrated for its relational data integrity, ACID guarantees, composite indexing, and granular access controls.

2.1 Schema Upgrade & Relational Normalization (3NF)

The schema in `migrations/001_initial_supabase_postgres_schema.sql` elevates the academic database to strict Third Normal Form (3NF), defining explicit Primary Keys (PK), Foreign Keys (FK), composite unique constraints, and stored generated columns:

- **`students`**: `student_id SERIAL PRIMARY KEY`, `name VARCHAR(150) NOT NULL`, `division VARCHAR(5)`, `user_id UUID REFERENCES users(user_id)`.
- **`subjects`**: `subject_id SERIAL PRIMARY KEY`, `name VARCHAR(100) UNIQUE`, `department VARCHAR(50)`, `credits INT CHECK (credits > 0)`.
- **`marks`**: `mark_id SERIAL PRIMARY KEY`, `student_id INT REFERENCES students(student_id) ON DELETE CASCADE`, `subject_id INT REFERENCES subjects(subject_id) ON DELETE CASCADE`, `marks NUMERIC(5,2) CHECK (marks ⩾ 0 AND marks ⩽ 100)`, composite unique key on `(student_id, subject_id, exam_type)`.
- **`attendance`**: Generated percentage column: `percentage NUMERIC(5,2) GENERATED ALWAYS AS (ROUND((attended::numeric / total_classes::numeric) * 100, 2)) STORED`.

- `query_logs`: Tracks query executions with `input_mode VARCHAR(10) CHECK (input_mode IN ('voice','text'))`, execution latency in milliseconds, row counts, and timestamps.

2.2 Row-Level Security (RLS) Policies

Supabase enforces granular authorization directly at the database kernel level using PostgreSQL Row-Level Security:

```
-- Enforce RLS on student profiles and marks
ALTER TABLE public.students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.marks ENABLE ROW LEVEL SECURITY;

-- Students view only their own records; Faculty and Admins view all
CREATE POLICY "Student read own profile"
  ON public.students FOR SELECT
  USING (
    auth.uid() IN (SELECT auth_uid FROM public.users WHERE users.user_id = students.user_id)
    OR EXISTS (SELECT 1 FROM public.users WHERE auth_uid = auth.uid() AND role IN ('faculty', 'admin
  );

CREATE POLICY "Student read own marks"
  ON public.marks FOR SELECT
  USING (
    student_id IN (
      SELECT s.student_id FROM public.students s
      JOIN public.users u ON s.user_id = u.user_id
      WHERE u.auth_uid = auth.uid()
    )
    OR EXISTS (SELECT 1 FROM public.users WHERE auth_uid = auth.uid() AND role IN ('faculty', 'admin
  );
```

3. Data Migration Strategy: SQLite to Cloud Postgres

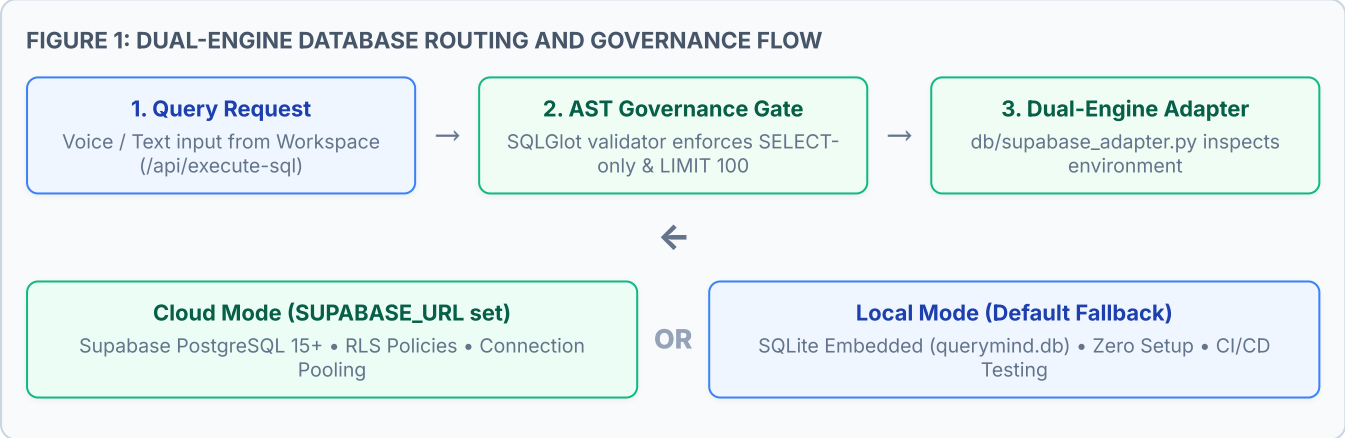
To ensure seamless transition from local SQLite to Supabase PostgreSQL, QueryMind implements a multi-path migration strategy guaranteeing referential integrity and type parity:

- **Automated Python Migration Utility (`scripts/migrate_sqlite_to_supabase.py`):** Connects directly to `db/querymind.db`, extracts rows from all academic tables, maps SQLite data types to PostgreSQL 15+ equivalents, derives attended/total class ratios, and emits idempotent SQL statements.
- **Transactional Seed Script (`migrations/002_data_seed_supabase.sql`):** Generates 257 lines of clean SQL wrapped in a `BEGIN; ... COMMIT;` transaction. Features `ON CONFLICT DO NOTHING` / `DO UPDATE` clauses and synchronizes PostgreSQL sequences via `setval(pg_get_serial_sequence(...))`.
- **Automated Tooling Comparison (`pgloader`):** For enterprise or continuous production migrations, `pgloader` can stream SQLite databases directly into PostgreSQL over TCP, handling type transformations and indexes in a single command:

```
pgloader db/querymind.db postgresql://postgres:password@db.supabase.co:5432/postgres
```

4. Dual-Engine Database Adapter Architecture

The application utilizes a Unified Database Adapter (`db/supabase_adapter.py`) that abstracts the storage engine from application routes:



This dual-driver approach allows students and developers to clone and run the application instantly with zero external dependencies, while production deployments seamlessly switch to cloud PostgreSQL with complete RLS enforcement.

5. Advanced Relational DBMS Features & Pedagogy

QueryMind 2.0 integrates sophisticated relational database internals directly into the user interface:

- **Query Execution Plan Analyzer (`EXPLAIN QUERY PLAN`):** Exposed via `POST /api/explain-sql` and implemented in `explain_query_plan()`. Parses the DBMS execution tree to identify sequential scans (`FULL_TABLE_SCAN`), index searches (`INDEX_SEARCH`), and temporary B-Tree sorting (`TEMP_BTREE_SORT`). Displayed in an interactive dark-mode drawer on both the Workspace and SQL Playground.
- **Dynamic Entity-Relationship Diagram:** Uses live database catalog introspection (`get_database_schema_metadata()`) to dynamically render interactive Mermaid ER diagrams reflecting foreign key relationships.

- **Deterministic AST Security Governance:** Employs **SQLGlot** to parse SQL into an Abstract Syntax Tree, strictly permitting `SELECT` expressions, rejecting all DDL/DML tokens (`DROP` , `INSERT` , `DELETE`), blocking comments and multi-statement semicolons, and automatically appending a `LIMIT 100` safety ceiling.
- **Auditing & Input Modality Telemetry:** Records all queries into `query_logs` with millisecond execution latency, row count, user attribution, and an explicit `input_mode` badge (`🎙 Voice` vs `⌨ Text`).

6. Security, Governance & Verification Architecture

The system implements defense-in-depth across the web and database layers:

Security Layer	Implementation Mechanism	Threat Neutralized
AST Whitelist Gateway	SQLGlot parser in `nlp/sql_validator.py`	Blocks SQL injection, DDL drops, data tampering, multi-statement injection, and comment bypasses.
Row-Level Security (RLS)	PostgreSQL kernel policies in Supabase	Guarantees multi-tenant data isolation; students cannot query other students' marks or profiles.
Resource Ceilings	Automatic `LIMIT 100` AST injection	Prevents server memory exhaustion and Denial-of-Service via massive table dumps.
Read-Only Connections	SQLite `mode=ro` & unprivileged Postgres roles	Ensures voice queries cannot mutate persistent database state even if application logic is bypassed.

7. Automated Testing & Verification Standard

QueryMind 2.0 enforces a strict Test-Driven Development (TDD) protocol: $$\mathbf{Implement} \longrightarrow \mathbf{Automated\ Test} \longrightarrow \mathbf{Identify\ Failures} \longrightarrow \mathbf{Refactor\ \&\ Fix} \longrightarrow \mathbf{Retest\ to\ 100\%}$$ **Zero Unverified Claims Policy:** Every claimed capability must be validated by executable automated test cases. The repository currently maintains **59 automated tests** across 4 test suites, all passing with zero errors:

- `tests/test_querymind.py` (23 tests): Rule-based NL-to-SQL pattern matching, AST validator, CSV dataset ingestion.
- `tests/test_querymind_2.py` (26 tests): Spoken number normalization, 3-table joins, threshold filters, async REST endpoints, all 6 page routes.
- `tests/test_supabase_adapter.py` (3 tests): Database adapter initialization, fallback routing, and live catalog schema introspection.
- `tests/test_advanced_dbms.py` (7 tests): `EXPLAIN QUERY PLAN` parser, index search detection, B-tree sorting, injection blocking on explain endpoints, and SQLite-to-Supabase migration generation.

8. Autonomous Development Loop Prompt

AUTONOMOUS CODING AGENT EXECUTION PROMPT

System Role: You are an expert Principal Full-Stack Engineer and Relational Database Systems Architect. Your mission is to maintain and extend QueryMind 2.0 (Voice-to-SQL with Supabase PostgreSQL).

Starting Baseline: You are working on the `querymind-2.0` branch with 59/59 passing tests, Web Speech API capture, dual-engine database adapter, and SQLGlot AST validation. Do not re-architect existing working modules.

Autonomous Loop Protocol:

1. Read requirement and inspect existing code.
2. Implement targeted changes using clean, documented Python/JS.
3. Immediately run `pytest tests/ -v`. If any test fails, stop and inspect the traceback.
4. Refactor root causes and retest until 100% of tests pass.
5. Commit passing code with clear commit messages and push to `querymind-2.0`.

Non-Negotiable Constraints: Microphone is strictly an input mechanism; never auto-execute queries on speech end. Always enforce read-only SELECT allow-lists and LIMIT 100 injection. Maintain zero-config SQLite fallback for local development.

9. Academic Report Outline (Curriculum Standard)

The corresponding academic monograph follows IEEE formal project documentation standards:

1. **Title & Abstract:** QueryMind 2.0: A Voice-Driven Relational Database Interface with Cloud PostgreSQL Architecture.
2. **Introduction & Problem Formulation:** Bridging conversational natural-language interfaces with rigorous DBMS concepts (schema governance, relational integrity, query processing).
3. **System Architecture:** Multi-tier architecture overview: Web Speech API & audio waveform client, Flask application server, SQLGlot AST security gateway, and Supabase cloud PostgreSQL.
4. **Relational Database Design & Normalization:** Mathematical formulation of Functional Dependencies; verification of Boyce-Codd / Third Normal Form (3NF); foreign keys, composite indexes, and generated columns.
5. **Speech Recognition & Voice Modality:** Client-side Web Speech API analysis; Vosk and OpenAI Whisper fallback tiers; spoken number normalization algorithms.
6. **Query Execution & Optimization Pedagogy:** Query execution tree parsing (`EXPLAIN QUERY PLAN`); sequential table scans vs index-accelerated B-tree searches; temporary sorting buffers.
7. **Security & Row-Level Access Governance:** SQLGlot AST token allow-listing; SQL injection mitigation; PostgreSQL kernel-level Row-Level Security (RLS) policies.
8. **Data Migration & Dual-Engine Strategy:** SQLite-to-PostgreSQL data translation, sequence alignment, and dual-driver adapter architecture.
9. **Verification & Test-Driven Development:** Multi-tier test suite analysis (59 passing tests); security fuzzing; execution latency metrics (< 20ms).
10. **Conclusion & Future Extensions:** Project achievements, limitations (browser STT variance), and future integration of local quantized LLMs.

10. References & Primary Sources

1. **Mozilla Developer Network (MDN):** *Web Speech API Specification & SpeechRecognition Guide*, W3C Community Group.
2. **PostgreSQL Global Development Group:** *PostgreSQL 15.0 Documentation: Row Security Policies, Generated Columns & Explain Plans*, postgresql.org/docs.
3. **Supabase Inc.:** *Supabase Architecture, Row Level Security (RLS), and Postgres Audit (pgaudit) Guides*, supabase.com/docs.
4. **Alpha Cephei:** *Vosk Speech Recognition Toolkit & WebAssembly Runtime*, Apache 2.0 License, alphacephei.com/vosk.
5. **Radford, A., et al.:** *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*, OpenAI, arXiv:2212.04356.
6. **Mao, Toby:** *SQLGlot: A Modular Python SQL Parser, Transpiler, Optimizer, and Engine*, github.com/tobymao/sqlglot.
7. **Dimitri Fontaine:** *pgloader: Open Source Data Migration Tool for PostgreSQL*, pgloader.io.
8. **OWASP Foundation:** *SQL Injection Prevention Cheat Sheet & Defense in Depth Principles*, cheatsheetseries.owasp.org.